

B1 Series B

Setting up your environment and your first UMAP

Install today, plot today. The details get repaid over the next seventeen episodes.

About 30 min | Prerequisites: Series A | Data: PBMC 3k

Two ways to learn this pipeline

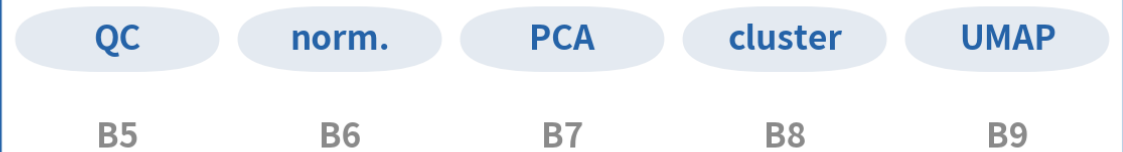
copy-pasting a tutorial

```
pbmc <- NormalizeData(pbmc)
pbmc <- RunPCA(pbmc)
pbmc <- FindClusters(pbmc,
  resolution = ???)
pbmc <- RunUMAP(pbmc, dims = ???)
```

**Error: change one thing, it breaks —
and you don't know why**

by the end of Series B

the same pipeline, but every
parameter you can defend: "why this value"



today: run the whole thing once

Left: copy-paste, breaks on any change. Right: this series' goal — every parameter defensible

THIS EPISODE DOES ONE THING

**Install everything today —
and plot your first UMAP today.**

Today covers only the what; every why gets a signpost to the episode that explains it.

What you will learn

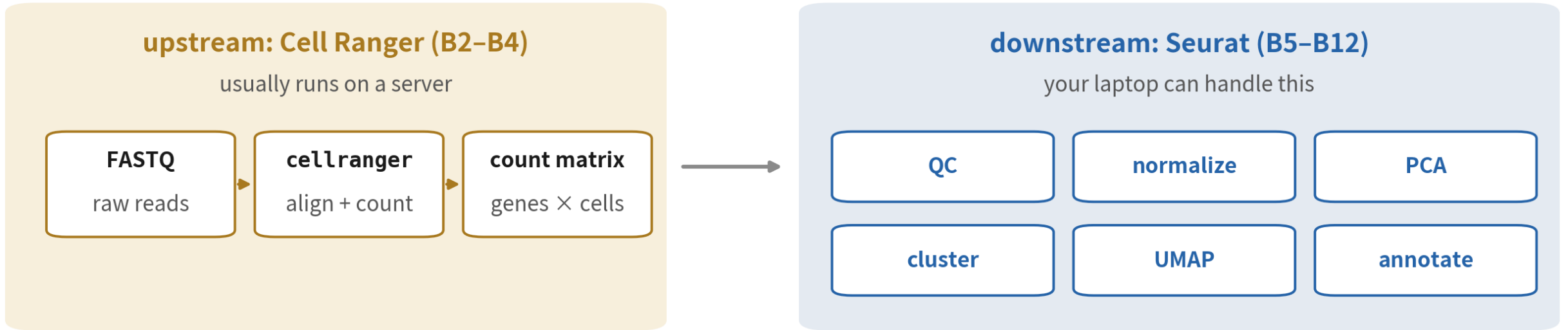
- 1 Install R, RStudio and Seurat v5, and verify the installation
- 2 Set up the series-wide project structure and renv — and say why versions get locked
- 3 Run PBMC 3k from file to UMAP in 15 lines, knowing roughly what each line does

01

The map before the territory

See the whole pipeline first, then choose your route

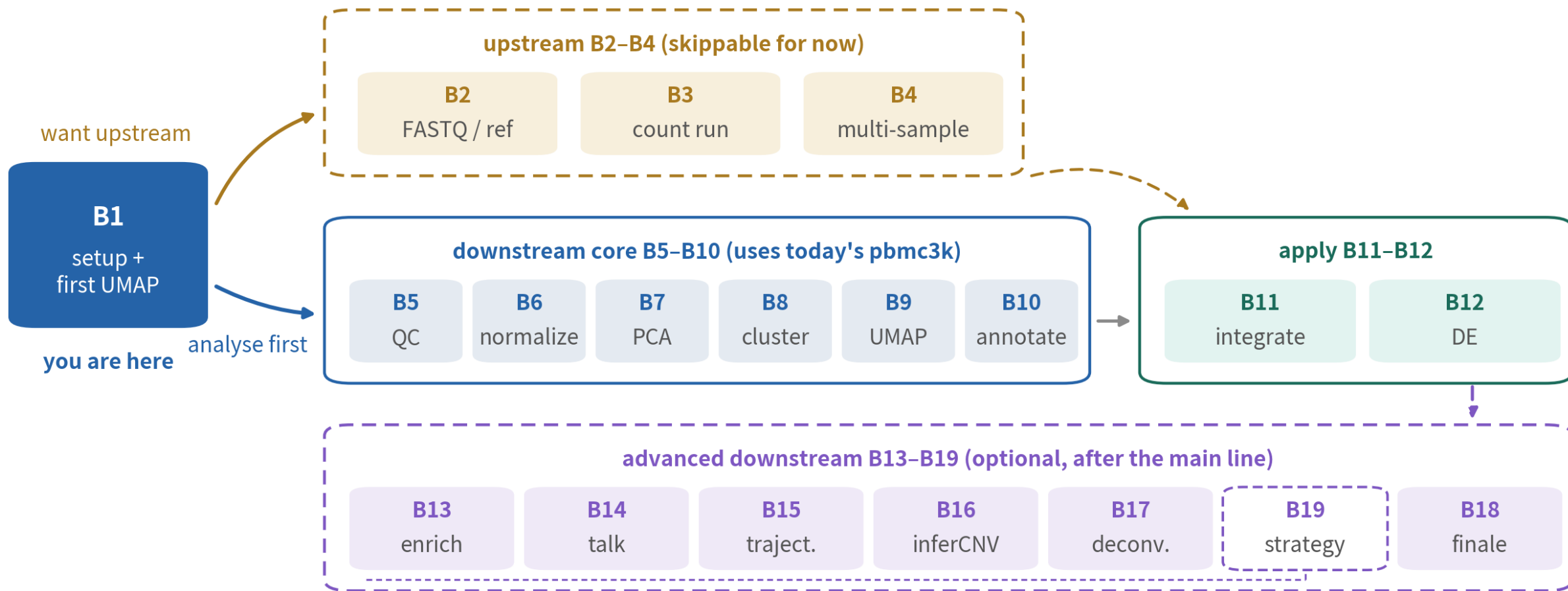
The toolchain: upstream and downstream



this episode: take a ready-made matrix and run the whole downstream once

Upstream turns reads into a matrix; downstream turns the matrix into biology — today we start from the matrix

The nineteen-episode map



both routes work: skipping B2-B4 does not block B5; pick B13-B19 as needed (B19 is the cross-cutting strategy capstone) — the series closes at B18

B2-B4 cover upstream and can wait; the B5-B12 main line is the spine; B13-B19 are optional advanced stops (B19 is the cross-cutting strategy capstone). Next: B5

LESSON ZERO OF THIS SERIES

**Reproducibility is not a virtue —
it is survival for your paper.**

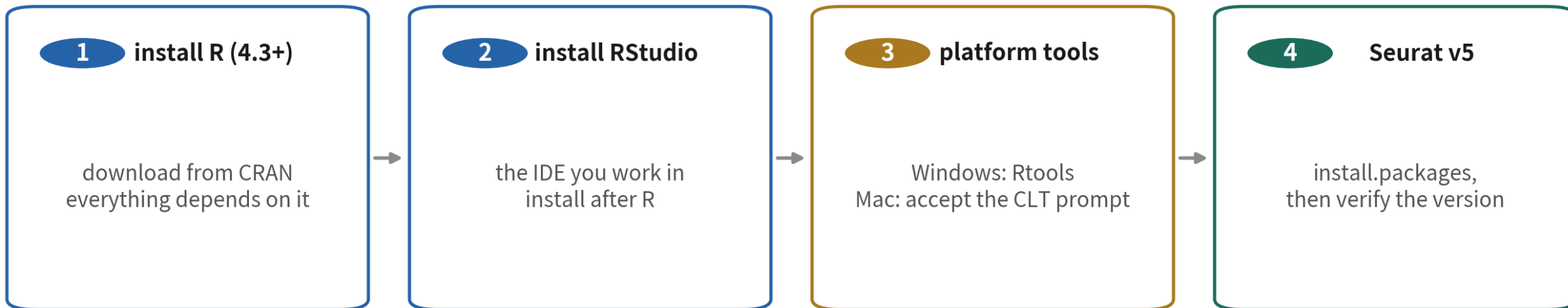
Same script, same data, same figure two years later. That is why lesson one is versions and seeds, not algorithms.

02

Install and verify

R → RStudio → platform tools → Seurat v5 — in that order

Installation order



order matters: RStudio needs R already installed; without Rtools many packages fail to compile

Four steps, five minutes each; if already installed, just run the final check

Three platform notes

Windows: Rtools



the compiler toolbox;
match it to your R
version (R 4.4 →
Rtools44)

Mac: command line tools



a prompt appears on
first install — accept it
and wait

R version 4.3+



required by Seurat v5;
if older, reinstall R —
do not fight it

Install Seurat and friends

```
# CRAN 上的 Seurat 現在就是 v5.x  
install.packages("Seurat")
```

```
# 本集另外會用到的兩個  
install.packages(c("tidyverse", "renv"))
```

```
# Windows 若提示需要 Rtools：先裝 Rtools，重開 RStudio 再跑  
# Mac 若跳出 command line tools 安裝視窗：同意，等它裝完
```

First install takes minutes; red text is not always an error — check the last line for 'error'

Verify the installation

```
library(Seurat)  
packageVersion("Seurat")  
R.version.string
```

```
Loading required package: Seurat  
[1] '5.1.0'  
[1] "R version 4.4.1 (2024-06-14)"
```

What you check: Seurat starts with 5, R is at least 4.3 — both, or it does not count

After section one you have

A working R setup



R 4.3+, RStudio and
platform tools in place

Seurat v5



version verified with
your own eyes, not
assumed

The one-line check habit



run `packageVersion()`
before ever asking for
help

03

Project structure and renv

Where files live and how versions lock — said once for the whole series

The project layout used all series

scrna-course/

- └─ `scrna-course.Rproj` project root
- └─ `renv.lock` always open via this
- └─ **`data/`** package version list
 - └─ `download_data.R` raw data: never edited
 - └─ `pbmc3k/` download script
- └─ **`R/`** extracted matrix
 - └─ `B01_first_umap.R` one script per episode
- └─ **`output/`** this episode's 15 lines
 - figures, rds, tables

all paths are relative

scripts only say `data/...`, `output/...`
move the folder anywhere, it still runs

data and code live apart

files in `data/` are never hand-edited
every change is a script in `R/`

outputs are disposable

delete `output/` entirely — rerunning
the scripts regrows it

data for raw files, R for scripts, output for products; every path is relative to the project root

Create the project and folders

```
# RStudio : File → New Project → New Directory → "scma-course"
```

```
# 之後每次都從 scma-course.Rproj 開專案
```

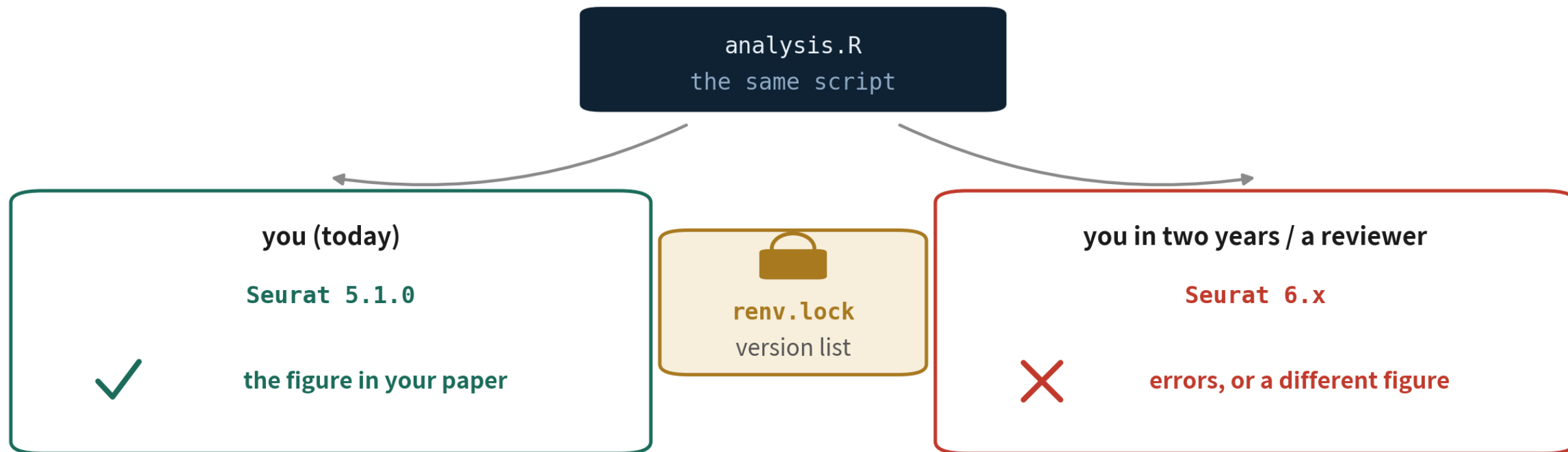
```
dir.create("data") # 原始資料：只進不改
```

```
dir.create("R") # 分析腳本：一集一支
```

```
dir.create("output") # 圖、rds、表格：可隨時重生
```

Opening the .Rproj sets the working directory to the root — why this series never calls setwd

Why lock versions at all



renv writes every package + version into `renv.lock`; on any machine `renv::restore()` rebuilds the same environment

Package updates change defaults, object structures, even results — `renv.lock` is your insurance

renv: insured in two calls

```
renv::init()    # 給這個專案一套獨立的套件庫  
  
# ... .. 套件都裝齊之後，任何時候：  
renv::snapshot() # 把目前的套件版本清單寫進  renv.lock
```

- Lockfile written to "~ /scma-course/renv.lock".

renv.lock travels with the project; renv::restore() rebuilds the same environment anywhere

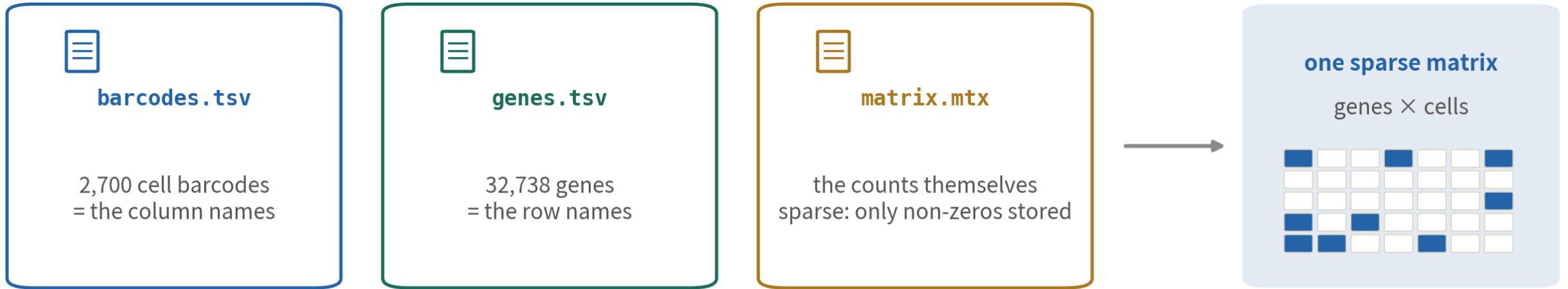
Download PBMC 3k: data/download_data.R

```
url<-paste0("https://cf.10xgenomics.com/samples/cell/",  
            "pbmc3k/pbmc3k_filtered_gene_bc_matrices.tar.gz")  
options(timeout= 600)      # 網速慢時不要中途放棄  
download.file(url, "data/pbmc3k.tar.gz", mode = "wb")  
untar("data/pbmc3k.tar.gz", exdir = "data/pbmc3k")  
list.files("data/pbmc3k", recursive = TRUE)
```

```
[1] "filtered_gene_bc_matrices/hg19/barcodes.tsv"  
[2] "filtered_gene_bc_matrices/hg19/genes.tsv"  
[3] "filtered_gene_bc_matrices/hg19/matrix.mtx"
```

About 7.6 MB; the three listed files are what success looks like

What you just downloaded



Read10X takes the folder holding these three files and reassembles the matrix (details in B3)

10x's official filtered matrix: 2,700 PBMCs — easy work for a laptop

After section two you have

A standard project



data/ R/ output/
plus .Rproj — used all
series

renv.lock



your environment has
a snapshot to restore
from

The pbmc3k data



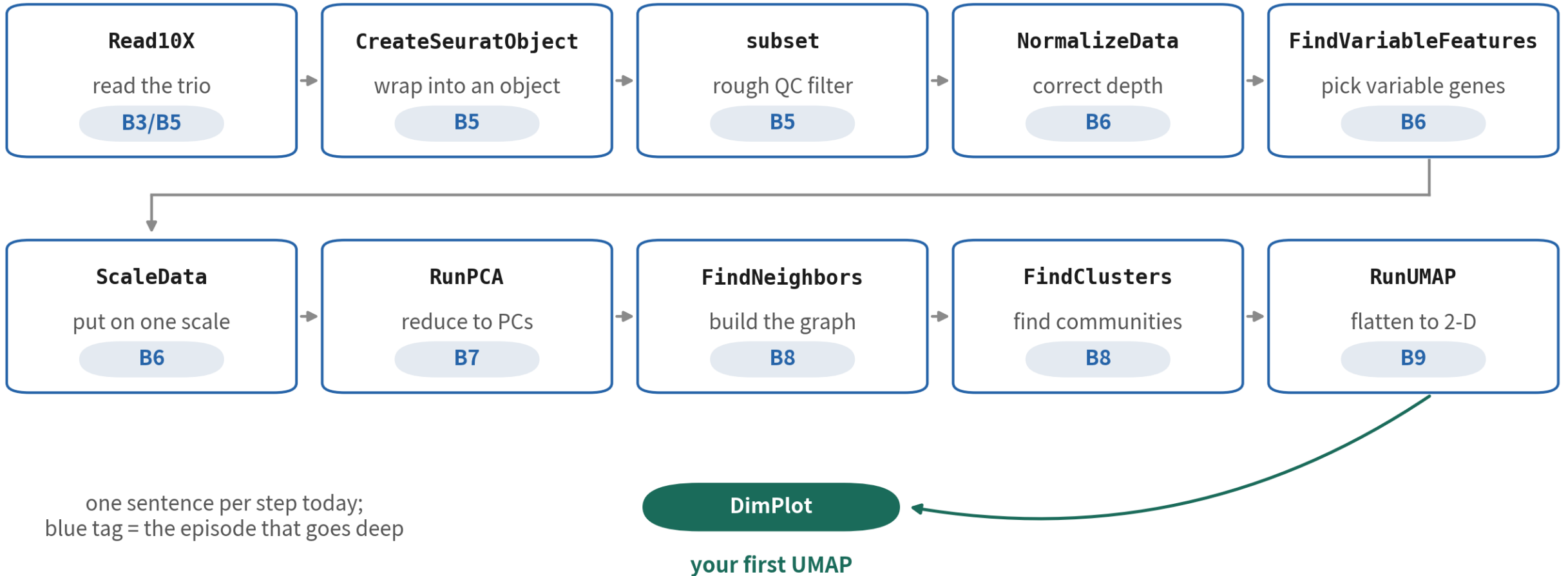
three files in place,
reused from B5
through B10

04

Fifteen lines to your first UMAP

One sentence per step; the whys are all signposted

The whole run on one picture



Ten functions chained into DimPlot; each blue tag names the episode that goes deep

Part 1: load, build, rough QC

```
library(Seurat)
set.seed(1234)           # 全系列固定的隨機種子

pbmc.data <- Read10X(data.dir =
  "data/pbmc3k/filtered_gene_bc_matrices/hg19/") # → B3/B5
pbmc <- CreateSeuratObject(counts = pbmc.data,
  project = "pbmc3k",
  min.cells = 3, min.features = 200)
pbmc[["percent.mt"]] <- PercentageFeatureSet(pbmc,
  pattern = "^MT-")
pbmc <- subset(pbmc, subset = nFeature_RNA > 200 &
  nFeature_RNA < 2500 & percent.mt < 5) # → B5
pbmc
```

An object of class Seurat
13714 features across 2638 samples within 1 assay

2,700 → 2,638: rough QC removes 62 cells. Copied thresholds today; B5 teaches you to set your own

Part 2: normalize through PCA

```
pbm c <- NormalizeData(pbm c)      # 校正定序深度    → B6  
pbm c <- FindVariableFeatures(pbm c) # 挑 2000 個 HVG → B6  
pbm c <- ScaleData(pbm c)          # 壓到同一尺度    → B6  
pbm c <- RunPCA(pbm c)              # 降到主成分空間  → B7
```

All four on defaults. Every default hides an assumption — B6 and B7 unpack them one by one

Part 3: cluster and UMAP

```
pbm c <- FindNeighbors(pbm c, dim s = 1:10) # 建鄰居圖 → B8  
pbm c <- FindClusters(pbm c, resolution = 0.5) # 找社群 → B8  
pbm c <- RunUMAP(pbm c, dim s = 1:10) # 壓成二維 → B9
```

```
Number of nodes: 2638  
Number of edges: 95965  
Running Louvain algorithm ...  
Maximum modularity in 10 random starts: 0.8723  
Number of communities: 9
```

The line to read is the last one: 9 communities means 9 clusters

Part 4: plot and save

```
DimPlot(pbm c, reduction = "um ap", label= TRUE)

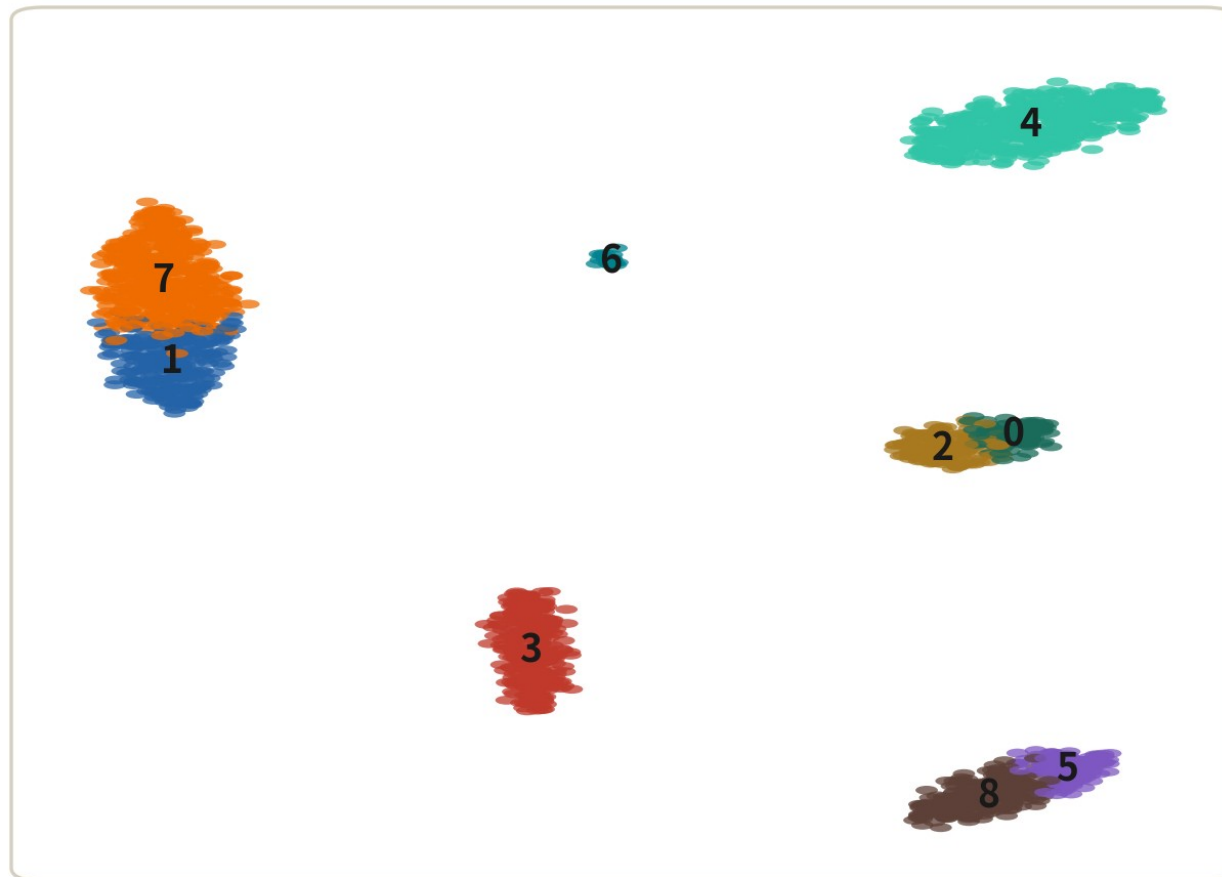
ggplot2::ggsave("output/B01_first_um ap.png",
                width = 7, height= 5.5, dpi= 200)
saveRDS(pbm c, "output/pbm c_b01_first_run.rds")
sessionInfo() # 版本紀錄，貼進你的分析筆記
```

Save the rds, save the plot, log sessionInfo — the same three-step wrap-up ends every episode

Your first UMAP

2,638 cells
9 clusters

one dot = one cell
colour = its cluster



who are these clusters?
T cells? B cells?
→ B10 answers that

why these
parameter values?
→ B5–B9

simulated data (live run uses pbmc3k)

2,638 cells, 9 clusters. Congratulations — you have run the whole downstream pipeline once

HOW TO FEEL ABOUT THIS PLOT

**First make it run, then make it make sense.
Every parameter today was a default.**

Running it gives you confidence and the big picture; understanding it is B5–B9's job. You need both, in this order.

Series A vocabulary, fast

barcode

each droplet's ID — one matrix column

UMI

molecule tag: count molecules, not reads

count matrix

the genes-by-cells starting table

doublet

two cells in one droplet, posing as one

cluster

cells grouped by similar expression

UMAP

a 2-D map — for looking, not measuring

all six come from Series A; from here on they are used without re-teaching

These six get used without re-teaching; revisit Series A if any feels rusty

05

Where people get hurt

Both traps are environmental — and both get demonstrated for real

Trap 1: ignore versions, run whatever you copied

What it is

Copying an online tutorial without noticing it was written for Seurat v4.



Why it happens

Tutorials carry no expiry date, and search results favour old, popular posts.

What it costs you

v5 objects have no @counts slot — instant error; worse are the cases that run but behave differently.

Same line: fine in v4, explodes in v5

a v4-era tutorial

```
counts <- pbmc@assays$RNA@counts  
# v4: matrix in the @counts slot
```

run under v5:

**Error: no slot of name "counts" for
this object of class "Assay5"**

the Seurat v5 way

```
counts <- LayerData(pbmc,  
                    layer = "counts")  
# v5: matrices live in layers
```

counts

data

scale.data

layers: raw, normalized and scaled coexist

check the date on any tutorial: pre-2023 usually means v4 syntax — translate before you paste

v5 moved matrices into layers; check the publication date before trusting a tutorial

Trap 2: hard-coding paths with setwd()

What it is

Starting every script with `setwd("C:/Users/your-name/...")`.



Why it happens

It feels like the obvious fix — hard-code the path and today's error goes away.

What it costs you

Any other machine — a labmate's, or yours in two years — dies on line one; non-ASCII user names break some packages' file reads too.

Absolute paths vs the RStudio Project

hard-coded absolute path

```
setwd("C:/Users/ming/Desktop/  
analysis_final_v2")
```

on any other machine or account:

```
Error in setwd(...) :  
cannot change working directory
```

RStudio Project + relative paths

```
# open .Rproj: wd = project root  
Read10X("data/pbmc3k/...")
```

move the whole folder anywhere:



runs unchanged — not one edit

house rule: no setwd(), no absolute paths — every script in this series follows it

Project + relative paths: move the folder anywhere, change nothing

Other install traps, quick reference

R too old



'requires R >= 4.x' → reinstall R; do not hunt for old packages

Non-ASCII paths



Chinese folder or user names → keep projects on an ASCII path like C:/scrna

Out of memory



pbmc3k needs just 8GB; only bigger data needs more RAM or a server

GitHub installs fail



install remotes first; if a campus network blocks GitHub, switch networks

TRAPS, SUMMED UP

**Environment pain is not a talent problem —
it is everyone's first boss fight.**

Verify versions, use relative paths, lock with renv — and nine tenths of the 'mystery errors' never happen.



Strategy Room: Complex Samples

The standard pipeline is done — now imagine the sample in your hands is not PBMC but a tumor

Two tracks: walkthrough + strategy room

Complex samples: tumors, inflamed tissue, multi-region designs, blood cancers; strategy means decisions, not a new pipeline

The walkthrough, untouched



the pbmc/ifnb pipeline
stays as is — run it
through, then
understand it

Strategy room: same step, different call



2–5 slides at each
episode's end; no new
tools — which
decisions change on
complex samples, and
why

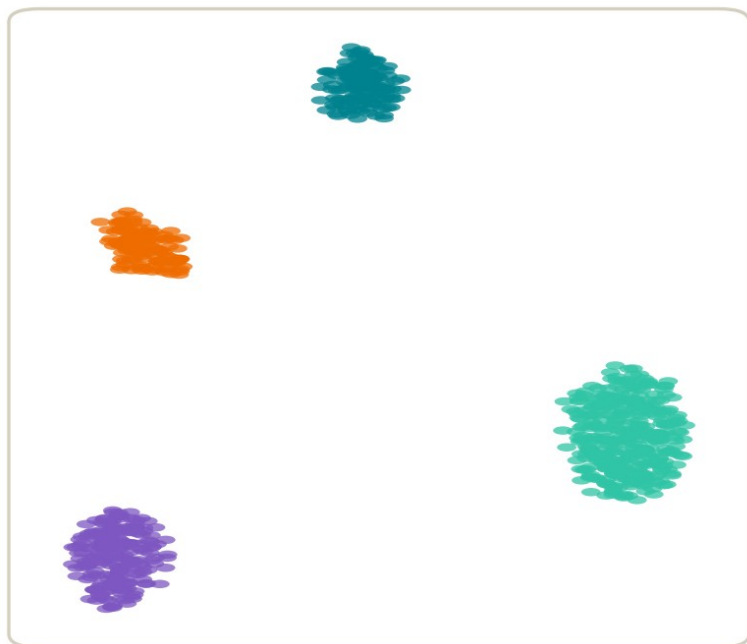
B19: different tumors, different playbooks



the strategies collected
along the way
assemble into one full
episode on strategy
design

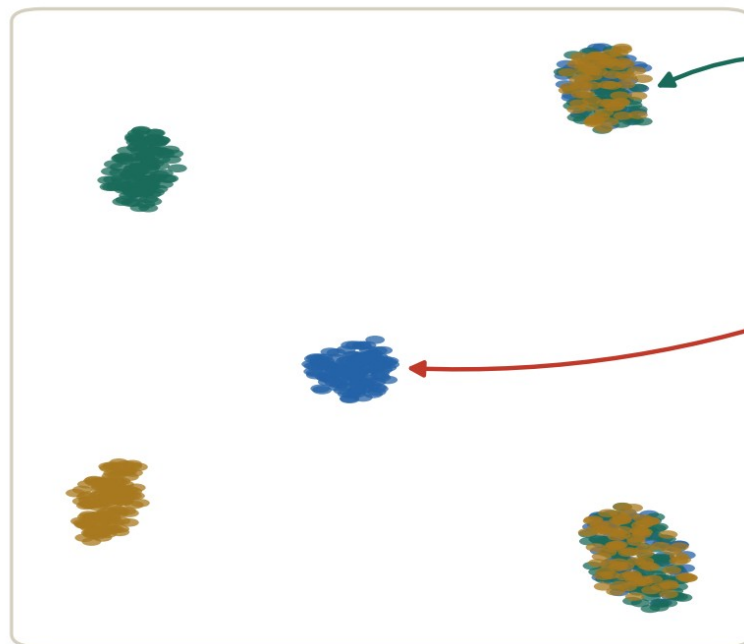
The same fifteen lines, fed a tumor

PBMC: quickstart-land



color = cell type; one cluster $\approx$ one type

a tumor: the same 15 lines



● patient 1 ● patient 2 ● patient 3

immune / stromal cells:
three patients mixing — fine

malignant cells:
one island per patient

normal cells can hide inside an island
markers can't tell → CNV (B16)

the code still runs, the plot still looks fine — but what a cluster means has changed; the strategy room defuses these reefs episode by episode

simulated data

The code still runs and the plot still looks fine — but malignant cells island by patient, malignant and normal can share a cluster, and the patient is the statistical unit

Episode checklist



R 4.3+ and RStudio installed and launching



`packageVersion("Seurat")` reports 5.x



project ready: data/ R/ output/ plus `renv.lock`



the pbmc3k trio sits in data/pbmc3k/



R/B01_first_umap.R runs end-to-end with zero errors



output/ holds your first UMAP and the .rds

All six ticked and you are ready for B5 (or B2)

ONE-LINE SUMMARY

**Your environment is part of the experiment:
verify versions, keep paths relative, fix the seed.**

Every parameter behind today's UMAP was a default — hold that thought for B5.

Check yourself 1

Why does this series demand renv version-locking in episode one?

- A renv makes code run faster
- B Package updates can change defaults and results; locking makes the analysis reproducible
- C Seurat cannot be installed without renv
- D renv fixes coding errors automatically

Answer B. The same script can error out — or give different results — under different versions. renv.lock records every package version so that you in two years, a labmate, or a reviewer can rebuild the identical environment.

Check yourself 2

What should the argument of `Read10X()` point to?

- A The matrix.mtx file
- B The downloaded tar.gz archive
- C The folder holding the three files (barcodes, genes, matrix)
- D The 10x download URL

Answer C. `Read10X` takes a folder path and reassembles barcodes.tsv, genes.tsv and matrix.mtx into one sparse matrix itself. Pointing it at a single file or the archive errors out — the most common first error message.

Check yourself 3

A friend's script starts with `setwd("C:/Users/ming/Desktop/analysis")` and dies on your machine. The right fix?

- A Change it to your own absolute path
- B Use an RStudio Project, delete the `setwd`, switch to relative paths
- C Recreate his folder structure on your machine
- D Run RStudio as administrator

Answer B. Swapping in your own absolute path (A) just hands the same landmine to the next person. The fix is a Project plus relative paths: opening the `.Rproj` sets the working directory to the root, so `data/` and `output/` hold on any machine.

NEXT EPISODE

B2: References and FASTQ (want analysis first? jump to B5)

Every step behind today's UMAP ran on defaults — are the defaults right? Says who?

Upstream-curious viewers go to B2 to see where the matrix comes from; analysts jump to B5 and start interrogating the first QC cutoff.